# BusinessOPS and analyticsDASHBOARD

**FINAL PROJECT SPECIFICATION & ANTIGRAVITY BUILD GUIDE**

**Architecture: Java 21 · Spring Boot · Firebase Authentication · Cloud Firestore**

**Frontend: HTML5 · CSS3 · JavaScript | Charts: Chart.js**

## Purpose

A practical, enterprise-style business operations dashboard for managing clients, income, expenses, and business performance. Designed for a college demonstration, internship portfolio, and possible future MVP.

**Repository rule:**
Use the existing GitHub repository. Do not create a new repository. Inspect and preserve existing work before changing files.

Prepared: 29 October 2026

# 1. Executive summary

Build a full-stack business operations and analytics application with a straightforward, testable architecture. The frontend provides the user interface; Firebase Authentication handles identity; a Java Spring Boot REST API verifies Firebase ID tokens, applies business rules, and accesses Cloud Firestore through the Firebase Admin SDK.

> **This keeps Firebase for both authentication and database storage while adding a Java backend. It does not use Next.js, TypeScript, PostgreSQL, or a separate Node.js/Express server.**

## Project outcomes

- Users can register, sign in, and access protected business data.
- A user can create a workspace and manage clients.
- Users can create, view, edit, delete, and filter income and expense records.
- The dashboard calculates KPIs and visualizes monthly trends and expense categories from saved data.
- The project includes validation, access controls, automated tests, setup instructions, and deployment configuration.

# 2. Final technology decisions

| Layer | Final choice | Purpose |
|---|---|---|
| Frontend | HTML5, CSS3, JavaScript ES6+ | Browser interface and REST API calls. |
| Backend | Java 21 + Spring Boot | Business logic, REST endpoints, validation, authorization. |
| Authentication | Firebase Authentication | Registration/sign-in; browser obtains Firebase ID tokens. |
| Database | Cloud Firestore | Stores workspaces, clients, transactions, categories. |
| Server Firebase integration | Firebase Admin SDK for Java | Verifies ID tokens and lets trusted server access Firestore. |
| Charts | Chart.js | Monthly trends and category breakdown. |
| Testing | JUnit 5, Mockito, Spring Boot Test | Unit and integration tests. |
| Version control | Existing GitHub repository | Tracks work; do not create another repository. |
| Hosting | Java-compatible backend host + static frontend host | Deploy the API and frontend. |

# 3. Architecture and request flow

Use one existing repository with a static frontend and a Spring Boot backend. Keep the design simple until the core user journey works.

```
Browser
  1. Firebase Authentication (Firebase JS SDK) signs user in
  2. Browser obtains Firebase ID token
  |
  v
Frontend (HTML / CSS / JavaScript)
  | HTTPS REST request + Authorization: Bearer <ID token>
  v
Spring Boot REST API (Java 21)
  | verify token with Firebase Admin SDK
  | derive authenticated UID; authorize workspace/record
  | validate input and apply business rules
  v
Cloud Firestore (accessed by trusted backend)
  |
  v
JSON response -> UI updates tables, KPI cards, and charts
```

> **Security warning: Firebase Admin SDK uses privileged server credentials and bypasses Firestore Security Rules. The backend must enforce authorization and ownership on every request. Do not assume Firestore rules protect Admin SDK operations.**

### Suggested repository structure

```
repository-root/
  frontend/
    index.html
    login.html
    dashboard.html
    clients.html
    transactions.html
    assets/css/
    assets/js/
  backend/
    src/main/java/<package-path>/
      config/  security/  controller/  service/
      repository/  model/  dto/  exception/
    src/main/resources/
    src/test/java/<package-path>/
  docs/
  README.md
  .gitignore
```

Antigravity must inspect the existing repository before applying this structure. Adapt a working structure rather than deleting it or duplicating folders.

## 4. MVP scope and priorities

| Priority | Features | Acceptance criteria |
|---|---|---|
| Tier 1 — must ship | Authentication; workspace; client CRUD; income/expense CRUD; filters; KPI dashboard; charts; responsive UI; access control; tests; README; deployment. | Core journey works with real Firestore data; users cannot access another workspace's records. |
| Tier 2 — after core is stable | CSV export/import with preview; invoices/payment status; date-range comparison; light/dark theme; rule-based alerts. | Each feature has validation, useful error states, and tests. |
| Tier 3 — future roadmap | Team invitations/roles; AI assistant; scheduled reports; integrations; subscription billing; forecasting. | Plan separately after MVP feedback; do not add to initial build. |

### Core user journey

- Register or sign in.
- Create a workspace and set display name/default currency (INR).
- Add a client.
- Record an income and an expense transaction.
- Open the dashboard and confirm KPIs, trends, category chart, and recent activity update.
- Edit/delete a record and confirm the dashboard recalculates.
- Sign out and verify protected pages/API endpoints reject anonymous access.

# 5. Cloud Firestore data model

Use a workspace-oriented model even if the first release is owner-first. It leaves room for team access later without requiring a complete redesign.

```
users/{uid}
  displayName, email, createdAt

workspaces/{workspaceId}
  name, ownerUid, defaultCurrency: "INR", createdAt, updatedAt

workspaces/{workspaceId}/members/{uid}
  role: "OWNER" | "ADMIN" | "MEMBER", joinedAt

workspaces/{workspaceId}/clients/{clientId}
  name, company, email, phone, status, notes, createdAt, updatedAt

workspaces/{workspaceId}/transactions/{transactionId}
  type: "INCOME" | "EXPENSE"
  amountMinor, currency, category, optional clientId
  description, transactionDate, createdByUid, createdAt, updatedAt

workspaces/{workspaceId}/categories/{categoryId}
  name, type: "INCOME" | "EXPENSE", isDefault, createdAt
```

## Data design rules

- Use document IDs, not display names, as identifiers.
- Store money in integer minor units where practical (INR paise); format values as rupees in the UI.
- Use consistent dates and server timestamps for audit fields where possible.
- Do not trust workspaceId, ownerUid, createdByUid, or role values sent by the browser; derive or verify them on the backend.
- Create required Firestore composite indexes for queries and document them.

# 6. Authentication, authorization, and security

Authentication asks who is signed in; authorization asks what that person is allowed to do. Both are required.

- The browser uses Firebase Authentication to sign in and obtains an ID token.
- Every protected API request includes Authorization: Bearer <ID token>.
- Spring Boot verifies the token and obtains the UID from the verified token.
- The service layer checks that the UID owns or belongs to the workspace and has the required permission.
- Only after authorization succeeds may the backend access the requested data.
- Use 401 for missing/invalid authentication, 403 for insufficient permissions, 404 for missing records, 400 for invalid input, and 500 for unexpected errors.

## Security checklist

- Never commit service-account JSON, private keys, passwords, tokens, or production credentials.
- Use environment variables or a hosting provider’s secret manager for server configuration.
- Never expose Firebase Admin credentials to the browser.
- Validate incoming data using DTOs and Jakarta Bean Validation.
- Enforce workspace access on reads, writes, deletes, exports, and analytics.
- Restrict production CORS to the deployed frontend origin.
- Configure Firestore rules defensively. If the browser does not access business collections directly, deny direct client access to those collections unless a use case requires otherwise.
- Do not log tokens, passwords, private keys, or sensitive customer data.
- Do not store user passwords in Firestore; Firebase Authentication handles passwords.

# 7. REST API outline

Use a consistent versioned prefix such as /api/v1. Endpoint names can be adapted to the existing repository, but keep naming consistent and document it.

| Method + endpoint | Purpose | Access |
|---|---|---|
| GET /api/v1/me | Return verified identity/profile basics. | Signed-in user |
| POST /api/v1/workspaces | Create workspace for authenticated user. | Signed-in user |
| GET /api/v1/workspaces | List accessible workspaces. | Signed-in user |
| GET /api/v1/workspaces/{id}/clients | List/search clients. | Workspace member |
| POST /api/v1/workspaces/{id}/clients | Create client. | Member with write permission |
| GET /api/v1/workspaces/{id}/clients/{clientId} | Read client. | Workspace member |
| PUT /api/v1/workspaces/{id}/clients/{clientId} | Update client. | Member with write permission |
| DELETE /api/v1/workspaces/{id}/clients/{clientId} | Delete client after dependency checks. | Owner/admin |
| GET /api/v1/workspaces/{id}/transactions | List/filter transactions. | Workspace member |
| POST /api/v1/workspaces/{id}/transactions | Create income/expense transaction. | Member with write permission |
| PUT /api/v1/workspaces/{id}/transactions/{txnId} | Update transaction. | Member with write permission |
| DELETE /api/v1/workspaces/{id}/transactions/{txnId} | Delete transaction. | Defined workspace permission |
| GET /api/v1/workspaces/{id}/analytics/summary | Return KPIs for date range. | Workspace member |
| GET /api/v1/workspaces/{id}/analytics/trends | Return monthly totals. | Workspace member |

### API design expectations

- Use request/response DTOs rather than exposing internal model objects without control.
- Return consistent JSON errors and field-level validation messages where useful.
- Paginate long lists; validate date ranges, transaction types, amounts, and workspace membership.
- Check record existence and authorization before returning or modifying it.

# 8. Dashboard metrics and definitions

| Metric | MVP definition |
|---|---|
| Total income | Sum of INCOME transaction amounts in the selected period. |
| Total expenses | Sum of EXPENSE transaction amounts in the selected period. |
| Net cash flow | Total income minus total expenses in the selected period. |
| Income/expense trend | Income and expenses grouped by month. |
| Expense by category | Expense amount totals grouped by category. |
| Recent transactions | Latest transactions ordered by an explicitly defined date/time field. |
| Active clients | Count of clients whose status is ACTIVE. |

### Accounting note

> **Call the metric “Net cash flow,” not “profit” or “recognized revenue.” Recorded income/expenses are not automatically equivalent to accrual-accounting profit, recognized revenue, taxes, or outstanding receivables. Add invoices and accounting concepts later if needed.**

Analytics must use real Firestore records, not hard-coded demo values. Keep date-range and aggregation logic in testable service classes. Handle empty datasets, month boundaries, and integer minor units correctly.

## 9. UI and experience requirements

- Navigation: Dashboard, Clients, Transactions, Reports/Analytics, Workspace Settings, Account/Sign out.
- Light mode is the default. Dark mode is Tier 2 and must not delay the MVP.
- Use readable typography, consistent spacing, accessible contrast, form labels, and keyboard focus states.
- Provide loading, empty, success, validation, and error states for major workflows.
- Use responsive tables/cards on mobile; avoid horizontal overflow.
- Charts need labels, useful tooltips/legends, and a clear no-data state.
- Display INR by default and keep currency as a workspace setting for future expansion.
- Do not add a brand name; keep the exact project title.

## 10. Step-by-step Antigravity implementation plan

> **Antigravity must work in controlled phases. After each phase, it summarizes changes, runs checks, reports failures, and waits for approval. Do not ask it to generate the whole app in one shot.**

### Phase 0 — Inspect and protect the repository

- Inspect repository tree, README, Git status, branches, build files, and existing implementation.
- Identify current framework and preserve useful code. Do not delete or overwrite substantial work without explaining and asking approval.
- Check Java/JDK availability; provide setup steps if missing.
- Report existing state, risks, proposed files, and a minimal plan. Do not edit before inspection report; wait for approval.

### Phase 1 — Establish Java backend

- Create/adapt backend/ as a Java 21 Spring Boot project.
- Add Spring Web, Spring Security, validation, Firebase Admin SDK, and testing dependencies. Add extras only when justified.
- Create packages for config, security, controller, service, repository, model, dto, and exception.
- Add health endpoint, centralized error handling, configuration placeholders, and README run instructions.
- Run automated tests and package build; fix failures before proceeding.

### Phase 2 — Firebase setup and authentication

- Document Firebase project setup, Email/Password provider, Firestore database, authorized domains, and web app config.
- Use Firebase JavaScript SDK for register/login/logout and ID token acquisition.
- Verify bearer ID tokens in Spring Boot with Firebase Admin SDK; implement protected /api/v1/me.
- Test valid, missing, invalid, and expired-token behavior.
- Never store passwords in Firestore.

## Phase 3 — Workspace model and authorization

- Implement workspace creation/listing, default INR currency, owner membership, and access checks.
- Derive UID from verified token; never trust UID or role supplied by browser.
- Test cross-workspace isolation for reads and writes.
- Document collection paths and required indexes.

## Phase 4 — Client management

- Implement client DTOs, validation, service logic, REST endpoints, search/status filter, and pagination if needed.
- Build list, create/edit form, details, search, and delete confirmation.
- Test CRUD, invalid fields, missing IDs, and unauthorized access.

## Phase 5 — Income and expense records

- Implement type, amountMinor, currency, category, optional client reference, description, transaction date, and audit fields.
- Implement CRUD and filters for type, date range, category, and client.
- Validate amount, date, type, category, and workspace access.
- Test that edits/deletes affect subsequent queries and analytics.

## Phase 6 — Analytics dashboard

- Implement testable services for income, expenses, net cash flow, monthly trends, category totals, and recent transactions.
- Expose protected analytics endpoints with workspace authorization.
- Build KPI cards and Chart.js visualizations using real API responses.
- Test date boundaries, empty data, months without records, and integer minor-unit calculations.

## Phase 7 — Optional features after core is stable

- Add CSV export first; import requires preview, validation, duplicate strategy, and explicit confirmation.
- Add invoices/payment status only after clients and transactions are reliable.
- Add dark mode, period comparisons, or rule-based alerts only if time permits.
- Defer AI, subscription billing, team invitations, and forecasting.

## Phase 8 — Quality, deployment, handoff

- Run unit/integration tests and complete end-to-end manual journey.
- Verify anonymous access is rejected and cross-workspace access is denied.
- Check browser console, API errors, mobile layout, empty states, and accessibility basics.
- Inspect staged files for secrets; commit .gitignore and setup docs.
- Document prerequisites, Firebase setup, environment variables, commands, tests, endpoints, deployment, and limitations.
- Deploy frontend/backend, configure HTTPS/CORS/domains, and smoke-test the deployed app.

# 11. Copy-ready Antigravity master prompt

Open the existing repository in Antigravity, then paste this prompt. It explicitly requires inspection before implementation and approval gates after each phase.

```
PROJECT TITLE: BusinessOPS and analyticsDASHBOARD

ROLE
Act as a senior Java/Spring Boot full-stack engineer and careful technical mentor. Build a reliable MVP in small, testable phases. Explain decisions clearly for a third-year CSE student.

NON-NEGOTIABLE RULES
1. Use the EXISTING GitHub repository opened in the workspace. Do not create a new repository, remote, or unrelated project.
2. Inspect Git status and repository structure first. Preserve useful existing work. Do not delete/overwrite substantial work without explaining and asking approval.
3. Use the exact title "BusinessOPS and analyticsDASHBOARD". Do not invent a product brand.
4. Backend: Java 21 + Spring Boot. Frontend: HTML5, CSS3, JavaScript ES6+. Authentication: Firebase Authentication. Database: Cloud Firestore.
5. Use Firebase JavaScript SDK in browser for auth and ID token acquisition. Use Firebase Admin SDK for Java on the trusted server to verify tokens and access Firestore.
6. Never expose Firebase Admin credentials in browser code. Never commit service-account keys, private keys, passwords, tokens, or production secrets.
7. Do not replace Firebase with another database. Do not introduce Next.js, TypeScript, Node/Express, PostgreSQL, or unrelated frameworks unless explicitly approved.
8. Default currency INR. Store money in integer minor units (paise) where practical. Label the primary calculation "Net cash flow," not accounting profit.
9. Implement only the current phase. After reporting results, stop and wait for approval before the next phase.
10. Never claim a build, test, or deployment passed unless you actually ran it and observed the result.

PHASE 0 - INSPECT FIRST
Inspect repository tree, README, Git status, existing files/configuration, and current project structure. Check Java/JDK version. Summarize what exists, what can be reused, risks, and a safe plan. List intended file changes. Do not edit until this report is complete; wait for approval to begin Phase 1.

TARGET ARCHITECTURE
Browser frontend (HTML/CSS/JavaScript) -> Firebase Authentication (browser SDK) -> Firebase ID token in Authorization: Bearer header -> Spring Boot REST API -> Firebase Admin SDK verifies token, derives UID, applies authorization/business rules -> Cloud Firestore. The frontend uses the REST API for business data. Do not allow browser writes to business collections unless justified and secured.

SUGGESTED STRUCTURE
frontend/ (HTML, CSS, JS, assets)  |  backend/ (src/main/java, src/main/resources, tests)  |  docs/, README.md, .gitignore
Adapt to existing structure instead of duplicating a working layout.

BACKEND DEPENDENCIES
Spring Boot with Spring Web, Spring Security, Jakarta Bean Validation, Firebase Admin SDK for Java, Spring Boot Test/JUnit 5/Mockito. Add libraries only when needed and explain why. Use secure environment-based configuration.

DATA MODEL
- users/{uid}: displayName, email, createdAt
- workspaces/{workspaceId}: name, ownerUid, defaultCurrency="INR", createdAt, updatedAt
- workspaces/{workspaceId}/members/{uid}: role, joinedAt
- workspaces/{workspaceId}/clients/{clientId}: name, company, email, phone, status, notes, createdAt, updatedAt
- workspaces/{workspaceId}/transactions/{transactionId}: type INCOME|EXPENSE, amountMinor, currency, category, optional clientId, description, transactionDate, createdByUid, createdAt, updatedAt
- workspaces/{workspaceId}/categories/{categoryId}: name, type, isDefault, createdAt
Derive UID from verified token. Never trust ownerUid, createdByUid, membership, or role supplied by browser. Admin SDK bypasses Firestore Security Rules, so backend authorization is mandatory for every request.

MVP ORDER
1. Safe repository inspection and setup.  2. Spring Boot scaffold, health endpoint, error handling, tests, README.  3. Firebase setup docs, browser authentication, token verification, protected /api/v1/me.  4. Workspace creation/listing and strict authorization.  5. Client CRUD/search/status filter.  6. Income/expense CRUD, validation, filters, optional client association.  7. Analytics: income, expenses, net cash flow, monthly trends, expense by category, recent transactions.  8. Responsive UI, validation, loading/empty/error/success states.  9. Automated tests, security checks, deployment instructions, README.
Only after core stability may you add CSV, invoices, comparisons, alerts, or dark mode. AI, billing, invitations, and forecasting are future work.

API GUIDANCE
Use /api/v1. Implement endpoints for /me, workspaces, clients, transactions, and analytics. Use DTOs, validate inputs, paginate long lists, return consistent JSON errors, and use appropriate status codes (400, 401, 403, 404, 500). Authorize reads as well as writes, including analytics and exports.

SECURITY
Verify Firebase ID tokens server-side. Enforce workspace owner/member and role checks. Test cross-workspace isolation. Keep secrets out of Git and logs. Restrict production CORS. Do not expose tokens or credentials in errors. Configure Firestore rules defensively, remembering Admin SDK bypasses them. Never store passwords in Firestore.

TESTING
Run automated tests and package builds at every backend phase. Test calculations, validation, token handling, workspace isolation, empty datasets, date boundaries, invalid payloads, missing records, unauthorized access, and analytics after edits/deletes. Do not claim a check passed unless it ran.

REQUIRED PHASE GATE AFTER EVERY PHASE
Report (1) what was implemented, (2) files created/changed, (3) commands actually run and exact pass/fail result, (4) failures/risks/remaining work, (5) manual verification steps, and (6) checkpoint/commit recommendation. Then stop and wait for approval.

FINAL HANDOFF
Provide README covering JDK 21 prerequisites, Firebase setup, secure credentials/environment variables, local commands, tests, Firestore model/indexes, API endpoints, deployment, security notes, known limitations, and future roadmap. Use existing repository only. Start with Phase 0 inspection; do not implement code until its report is complete.
```

## 12. Suggested delivery schedule

This is a target plan, not a reason to skip tests or security. Reuse existing work where possible; defer optional features if the core takes longer.

| Date | Target |
|---|---|
| 29 Oct 2026 | Inspect existing repository; check JDK; scaffold backend; document Firebase setup. |
| 30 Oct 2026 | Authentication/token verification, workspace model, access-control tests, client CRUD. |
| 31 Oct 2026 | Income/expense CRUD, filters, analytics service, dashboard charts, calculation tests. |
| 1 Nov 2026 | UI polish, integration tests, responsive layout, README, deployment setup. |
| 2 Nov 2026 | End-to-end verification, security review, bug fixes, freeze MVP. |
| 3 Nov 2026 | Contingency only; optional features only if core is stable. |

## 13. Release checklist

- Existing GitHub repository used; no accidental new repository/remote.
- Exact project title retained.
- Java 21 version documented.
- Backend starts locally; automated tests and package build pass.
- Firebase auth works for registration, login, logout, and invalid credentials.
- Protected endpoints reject missing/invalid ID tokens.
- Workspace authorization blocks cross-user data access.
- Client and transaction CRUD works with Firestore data.
- Dashboard values match records and selected date range.
- No credentials/tokens in Git, frontend files, or logs.
- Loading, empty, validation, error, and success states are handled.
- README explains local setup, Firebase, tests, deployment, and limitations.
- Deployed app passes a smoke test using a non-sensitive test account.

## 14. Setup concepts and commands

A JDK is needed to compile and run Java. Antigravity may edit files without it installed, but it cannot honestly verify a local build if the environment lacks it.

```
# Check installed Java runtime and compiler versions
java -version
javac -version
```

For Firebase setup, create/select a Firebase project, enable Email/Password in Authentication, create a Cloud Firestore database, and configure the frontend Firebase web app. Configure server credentials separately and securely. Never paste a service-account private key into frontend code or a public repository.

# 15. Official documentation

| Topic | Official resource |
|---|---|
| Spring Boot documentation | https://docs.spring.io/spring-boot/index.html |
| Spring Boot system requirements | https://docs.spring.io/spring-boot/system-requirements.html |
| Firebase Authentication for web | https://firebase.google.com/docs/auth/web/start |
| Firebase Admin SDK setup | https://firebase.google.com/docs/admin/setup |
| Cloud Firestore | https://firebase.google.com/docs/firestore |
| Firebase Security Rules | https://firebase.google.com/docs/rules |
| Chart.js documentation | https://www.chartjs.org/docs/latest/ |

## Final decision summary

**Project: BusinessOPS and analyticsDASHBOARD**
**Backend: Java 21 + Spring Boot**
**Authentication: Firebase Authentication**
**Database: Cloud Firestore**
**Frontend: HTML/CSS/JavaScript**
**Repository: existing GitHub repository only**
**Method: phased implementation with tests and approval gates**